

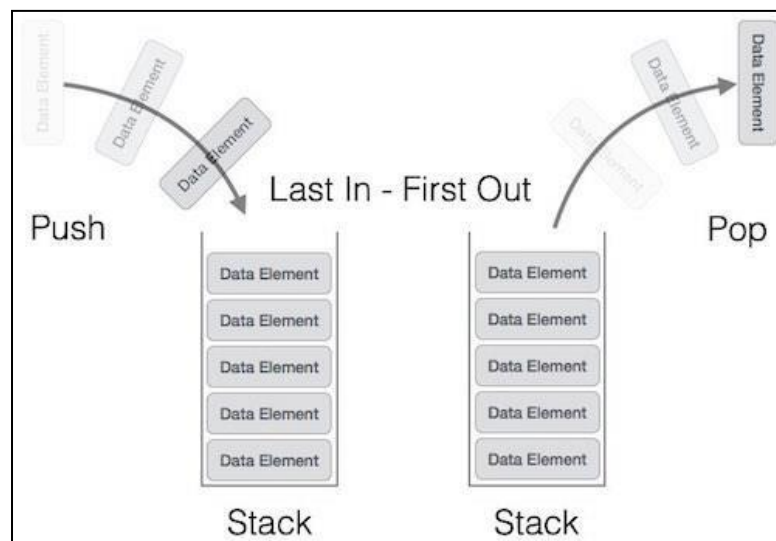
Unit 4: Stack

What is a Stack?

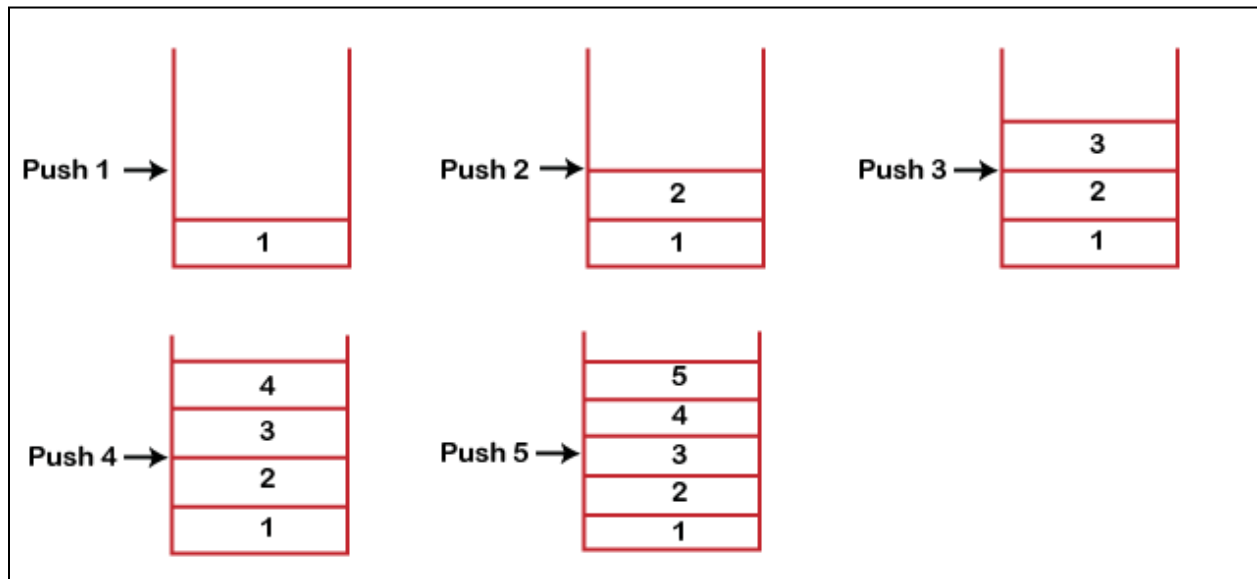
- **Def:** A stack is an ordered collection of homogeneous data elements where insertion and deletion operations take place at only one end called as Top of the Stack.
- A Stack is a linear data structure that follows the **LIFO (Last-In-First-Out)** principle where the last element inserted would be the first element deleted.
- It is named stack because it has the similar operations as the real-world stacks, for example – a pack of cards or a pile of plates, etc.



- Stack has one end. It contains only one pointer **top pointer** pointing to the topmost element of the stack.
- The position of the stack where operations are performed is called Top of the Stack. At any given time, we can only access the top element of a stack.
- Whenever an element is added in the stack, it is added on the top of the stack is called push() and the element can be deleted only from the stack called pop().



Working of Stack



Standard Stack Operations

The following are some common operations implemented on the stack:

- **init(S):** To initialize stack 'S';
- **push(S,x):** When we insert an element in a stack then the operation is known as a push. If the stack is full then the overflow condition occurs.
- **pop(S):** When we delete an element from the stack, the operation is known as a pop. If the stack is empty means that no element exists in the stack, this state is known as an underflow state.
- **isEmpty(S):** It determines whether the stack is empty or not.
- **isFull(S):** It determines whether the stack is full or not.'
- **peek():** It returns the element at the given position.

Time complexity of push(), pop() isEmpty(), isFull() in the worst case is $O(1)$ because stack work with one end of the data structure i.e Top of the Stack.

Representation of Stack:

1. Static representation using array
2. Dynamic representation using linked list

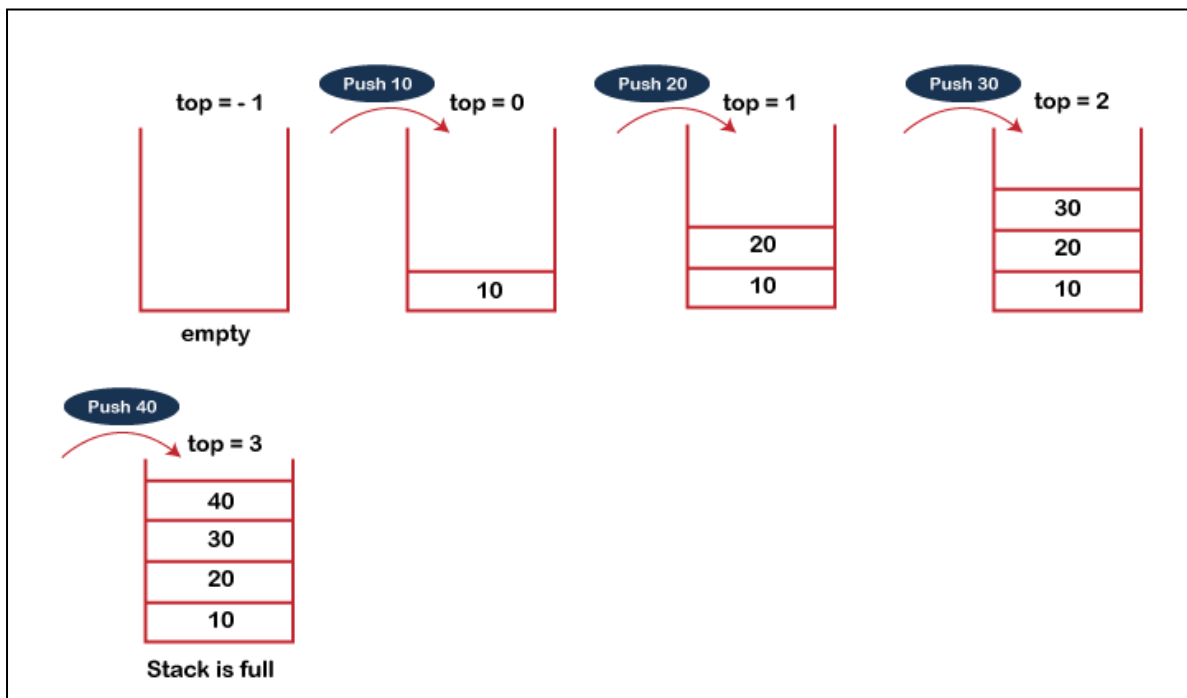
Implementation of Stack using array (Static Implementation)

- Array is used to store ordered list of elements.
- Limitation is imposed on the stack because stack is static.
- Once we declare the size it cannot be modified at runtime.

Push() :

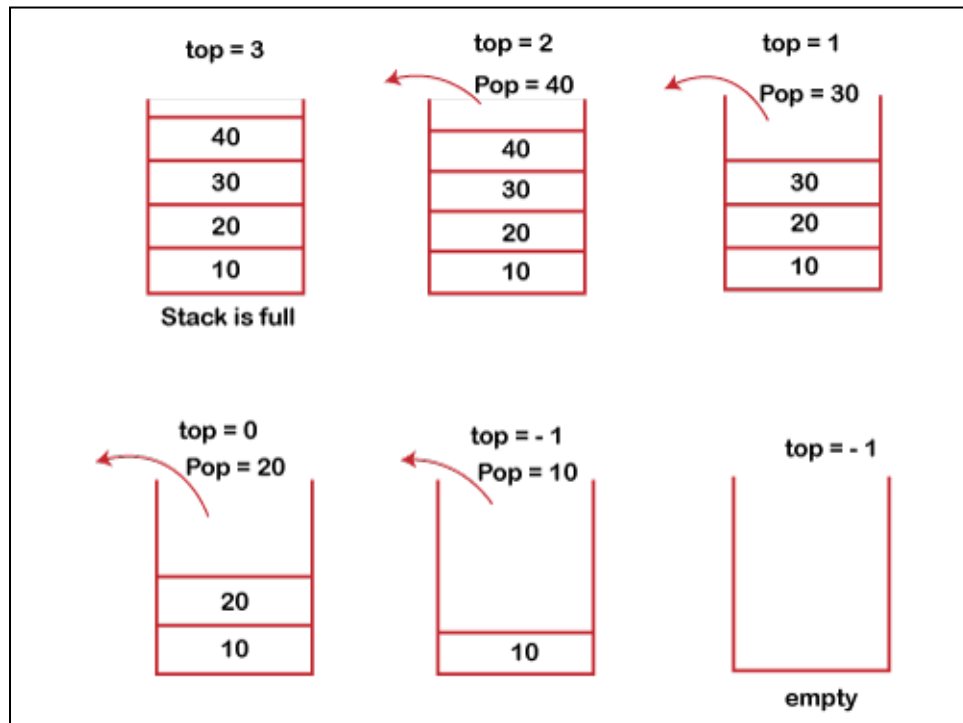
The steps involved in the PUSH operation is given below:

- Before inserting an element in a stack, we check whether the stack is full.
- If we try to insert the element in a stack, and the stack is full, then the **overflow** condition occurs.
- When we initialize a stack, we set the value of top as -1 to check that the stack is empty.
- When the new element is pushed in a stack, first, the value of the top gets incremented, i.e., **top=top+1**, and the element will be placed at the new position of the **top**.
- The elements will be inserted until we reach the **max** size of the stack.



POP(): The steps involved in the POP operation is given below:

- Before deleting the element from the stack, we check whether the stack is empty.
- If we try to delete the element from the empty stack, then the **underflow** condition occurs.
- If the stack is not empty, we first access the element which is pointed by the **top**
- Once the pop operation is performed, the top is decremented by 1, i.e., **top=top-1**.



Applications of Stack

The following are the applications of the stack:

- **String reversal:** Stack is also used for reversing a string. For example, we want to reverse a "Python" string, so we can achieve this with the help of a stack. First, we push all the characters of the string in a stack until we reach the null character. After pushing all the characters, we start taking out the character one by one until we reach the bottom of the stack.
- **UNDO/REDO:** It can also be used for performing UNDO/REDO operations. For example, we have an editor in which we write 'a', then 'b', and then 'c'; therefore, the text written in an editor is abc. So, there are three states, a, ab, and abc, which are stored in a stack. There would be two stacks in which one stack shows UNDO state, and the other shows REDO state.
If we want to perform UNDO operation, and want to achieve 'ab' state, then we implement pop operation.
- **Recursion:** The recursion means that the function is calling itself again. To maintain the previous states, the compiler creates a system stack in which all the previous records of the function are maintained.
- **DFS(Depth First Search):** This search is implemented on a Graph, and Graph uses the stack data structure.
- **Backtracking:** Suppose we have to create a path to solve a maze problem. If we are moving in a particular path, and we realize that we come on the wrong way. In order to come at the beginning of the path to create a new path, we have to use the stack data structure.

- **Expression conversion:** Stack can also be used for expression conversion. This is one of the most important applications of stack. The list of the expression conversion is given below:
 - Infix to prefix
 - Infix to postfix
 - Prefix to infix
 - Prefix to postfix
 - Postfix to infix
- **Memory management:** The stack manages the memory. The memory is assigned in the contiguous memory blocks. The memory is known as stack memory as all the variables are assigned in a function call stack memory. The memory size assigned to the program is known to the compiler. When the function is created, all its variables are assigned in the stack memory. When the function completed its execution, all the variables assigned in the stack are released.
- **Balancing of symbols:** Stack is used for balancing a symbol. For Example: $((a+b)*c)$. When the opening braces come, push the braces in a stack, and when the closing braces appear, pop the opening braces from the stack. Therefore, the net value comes out to be zero. If any symbol is left in the stack, it means that some syntax occurs in a program.

#Python Program to implement Stack operations using static implementation.

```
class Stack:
    def __init__(self, max_size):
        self.stack = []
        self.max_size = max_size

    def is_empty(self):
        return len(self.stack) == 0

    def is_full(self):
        return len(self.stack) == self.max_size

    def push(self, item):
        if self.is_full():
            print("Stack Overflow! Cannot push:", item)
        else:
            self.stack.append(item)
            print(f'Pushed {item} to stack.')

    def pop(self):
        if self.is_empty():
            print("Stack Underflow! Cannot pop from empty stack.")
            return None
        item = self.stack.pop()
        print(f'Popped {item} from stack.')
```

```

        return item

    def peek(self):
        if self.is_empty():
            print("Stack is empty. Nothing to peek.")
            return None
        return self.stack[-1]

    def display(self):
        print("Current Stack:", self.stack)

s = Stack(max_size=3)
s.push(10)
s.push(20)
s.push(30)
s.push(40) # Should show overflow message
s.display()
s.pop()
s.pop()
s.pop()
s.pop() # Should show underflow message
s.display()

```

String reversal Steps:

1. **Initialize a Stack:** Create an empty stack to hold the characters of the string.
2. **Push Characters onto the Stack:** Iterate through each character of the string and push it onto the stack.
3. **Pop Characters from the Stack:** Create an empty string to hold the reversed result. While the stack is not empty, pop the top character from the stack and append it to the result string.
4. **Return the Result:** The result string will now contain the characters of the original string in reverse order.

```

class Stack:
    def __init__(self, max_size):
        self.stack = []
        self.max_size = max_size

    def is_empty(self):
        return len(self.stack) == 0

    def is_full(self):
        return len(self.stack) == self.max_size

    def push(self, item):

```

```

        if self.is_full():
            print("Stack Overflow! Cannot push:", item)
        else:
            self.stack.append(item)

    def pop(self):
        if self.is_empty():
            print("Stack Underflow! Cannot pop from empty stack.")
            return None
        item = self.stack.pop()
        return item

    def display(self):
        print("Current Stack:", self.stack)

s=Stack(max_size=10)

str=input("Enter String:");
for i in range(len(str)):
    s.push(str[i]);
print("Reverse of String", str, "=", end="")
for i in range(len(str)):
    print(s.pop(), end="");

```

String reversal Using built in functions of stack

```

def reverse(s):
    # Step 1: Initialize a stack
    stack = []

    # Step 2: Push all characters onto the stack
    for char in s:
        stack.append(char)

    # Step 3: Pop characters from the stack to form the reversed string
    revStr = ""
    while stack:
        revStr = revStr + stack.pop()

    # Step 4: Return the reversed string
    return revStr

str = input("Enter the String:")
revStr = reverse(str)
print(revStr)

```

String reversal Using Slicing

```
def reverse(s):  
    return s[::-1]                                # slicing to reverse the string  
  
str = input("Enter the String:")  
print("Original String: ", str)  
print("Reverse of String: ", reverse(str))
```

Recursion: Python program to find factorial of a number using recursion.

```
def factorial(n):  
    if n == 0:                                    # Base case  
        return 1  
    else:  
        return n * factorial(n - 1)              # Recursive call  
  
n = int(input("Enter the Number:"))  
print("Factorial of n: ", factorial(n))
```

Recursion: Python program to print Fibonacci series using recursion.

```
def fibonacci(n):  
    if n == 0:  
        return 0  
    elif n == 1:  
        return 1  
    else:  
        return fibonacci(n-1) + fibonacci(n-2)  
  
print(fibonacci(10))
```